KNOWLEDGE ENGINEERING AND DATA SCIENCE

LABORATORY RECORD

Name	Dhuneshwaran V
Register Number	192324169
Course Name:	Knowledge Engineering and Data Science
Course Code	DSA1605
Lab Details:	Exp 1 To 3

Based on the executed Google Colab notebook and the supplied KE&DS Lab Manual

Table of Contents

Experiment 1: Data Exploration and Cleaning with Pandas (Simple)

Experiment 2: Semantic Network and Inheritance Queries (Simple)

Experiment 3: Frames and Inheritance (Simple)

Experiment 4: Rule-Based System with Forward Chaining (Simple)

Experiment 5: Backward Chaining with Why/How Explanation (Medium)

Experiment 6: Building and Reasoning over an Ontology (Medium)

Experiment 7: Building and Querying a Knowledge Graph (Medium)

Experiment 8: Hypothesis Testing, ANOVA and Chi-Square (Medium)

Experiment 9: Decision Tree Rule Learning (Hard)

Experiment 10: Hybrid Decision-Support System (Hard)

Experiment 1: Data Exploration and Cleaning with Pandas

Difficulty: Simple

Aim

To explore, clean and analyse a student marks dataset using Pandas, including missing values, duplicate records, invalid attendance values, summary statistics and visualisations.

Tools

Python, Google Colab, Pandas, NumPy, Matplotlib, Seaborn (as used in the notebook).

Procedure

1. Import the required Python libraries and load the student marks dataset.
2. Inspect the first records, dataset dimensions, data types and missing-value counts.
3. Clean missing values and duplicate records, and correct impossible attendance values.
4. Calculate descriptive statistics and department-wise averages.
5. Create the available plots and examine the correlation between attendance and final examination marks.
6. Save the cleaned dataset and review the recorded output.

Python Code

```
# EXPERIMENT 1: DATA EXPLORATION AND CLEANING WITH PANDAS

# 1. Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 2. Create student marks dataset
data = {
    'reg_no': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112],
    'department': ['AIDS', 'CSE', 'IT', 'AIDS', 'CSE', 'IT',
                   'AIDS', 'CSE', 'IT', 'AIDS', 'CSE', 'IT'],
    'gender': ['Male', 'Female', 'Male', 'Female', None, 'Male',
               'Female', 'Male', 'Female', 'Male', 'Female', 'Male'],
    'attendance': [90, 85, 75, 95, 80, 140, 88, None, 92, 78, 86, 91],
    'internal': [85, 80, 70, 90, 75, 65, 88, 82, 79, 72, 84, 89],
    'final_exam': [88, 82, 72, 94, 78, 68, 91, 85, None, 75, 87, 93],
    'study_hours': [4, 3, 2, 5, 3, 2, 5, 4, 3, 2, 4, 5]
}

df = pd.DataFrame(data)

# Add one duplicate record
df = pd.concat([df, df.iloc[[0]]], ignore_index=True)

# Save and load CSV file
df.to_csv('student_marks.csv', index=False)
df = pd.read_csv('student_marks.csv')

print("========== ORIGINAL DATASET ==========")
print(df)

# 3. Explore the dataset
print("\n========== FIRST FIVE RECORDS ==========")
print(df.head())

print("\n========== DATASET SHAPE ==========")
print(df.shape)

print("\n========== DATASET INFORMATION ==========")
df.info()

print("\n========== DESCRIPTIVE STATISTICS ==========")
```

```
print(df.describe())

# 4. Identify missing values
print("\n========== MISSING VALUES ==========")
print(df.isnull().sum())

# 5. Correct impossible attendance values
df.loc[df['attendance'] > 100, 'attendance'] = np.nan

# 6. Fill missing numerical values using median
numeric_columns = ['attendance', 'internal', 'final_exam', 'study_hours']

for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())

# 7. Fill missing categorical values using mode
categorical_columns = ['department', 'gender']

for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])

# 8. Remove duplicate records
df = df.drop_duplicates()

print("\n========== CLEANED DATASET ==========")
print(df)

print("\n========== MISSING VALUES AFTER CLEANING ==========")
print(df.isnull().sum())

print("\nFinal Dataset Shape:", df.shape)

# 9. Calculate statistical measures
print("\n========== FINAL EXAM STATISTICS ==========")
print("Mean:", df['final_exam'].mean())
print("Median:", df['final_exam'].median())
print("Standard Deviation:", df['final_exam'].std())
print("Minimum:", df['final_exam'].min())
print("Maximum:", df['final_exam'].max())

# 10. Department-wise average marks
print("\n========== DEPARTMENT-WISE MEAN ==========")
print(df.groupby('department')['final_exam'].mean())

# 11. Histogram
plt.figure(figsize=(7, 5))
plt.hist(df['final_exam'], bins=5, edgecolor='black')
plt.title('Distribution of Final Exam Marks')
plt.xlabel('Final Exam Marks')
plt.ylabel('Number of Students')
plt.show()

# 12. Box Plot
plt.figure(figsize=(7, 5))
df.boxplot(column='final_exam', by='department')
plt.title('Final Exam Marks by Department')
plt.suptitle('')
plt.xlabel('Department')
plt.ylabel('Final Exam Marks')
plt.show()

# 13. Scatter Plot
plt.figure(figsize=(7, 5))
plt.scatter(df['study_hours'], df['final_exam'])
plt.title('Study Hours vs Final Exam Marks')
plt.xlabel('Study Hours')
plt.ylabel('Final Exam Marks')
plt.show()

# 14. Correlation
print("\n========== CORRELATION ==========")
correlation = df['study_hours'].corr(df['final_exam'])
print("Correlation:", correlation)
```

```
# 15. Save cleaned dataset
df.to_csv('student_marks_clean.csv', index=False)

print("\nCleaned dataset saved successfully!")
print("EXPERIMENT 1 COMPLETED SUCCESSFULLY!")
```

Actual Output from the Executed Notebook

```
========== ORIGINAL DATASET ==========
    reg_no department  gender  attendance  internal  final_exam  study_hours
0      101       AIDS    Male        90.0        85        88.0            4
1      102        CSE  Female        85.0        80        82.0            3
2      103         IT    Male        75.0        70        72.0            2
3      104       AIDS  Female        95.0        90        94.0            5
4      105        CSE     NaN        80.0        75        78.0            3
5      106         IT    Male       140.0        65        68.0            2
6      107       AIDS  Female        88.0        88        91.0            5
7      108        CSE    Male         NaN        82        85.0            4
8      109         IT  Female        92.0        79         NaN            3
9      110       AIDS    Male        78.0        72        75.0            2
10     111        CSE  Female        86.0        84        87.0            4
11     112         IT    Male        91.0        89        93.0            5
12     101       AIDS    Male        90.0        85        88.0            4

========== FIRST FIVE RECORDS ==========
   reg_no department  gender  attendance  internal  final_exam  study_hours
0     101       AIDS    Male        90.0        85        88.0            4
1     102        CSE  Female        85.0        80        82.0            3
2     103         IT    Male        75.0        70        72.0            2
3     104       AIDS  Female        95.0        90        94.0            5
4     105        CSE     NaN        80.0        75        78.0            3

========== DATASET SHAPE ==========
(13, 7)

========== DATASET INFORMATION ==========
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13 entries, 0 to 12
Data columns (total 7 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   reg_no       13 non-null     int64
 1   department   13 non-null     object
 2   gender       12 non-null     object
 3   attendance   12 non-null     float64
 4   internal     13 non-null     int64
 5   final_exam   12 non-null     float64
 6   study_hours  13 non-null     int64
dtypes: float64(2), int64(3), object(2)
memory usage: 860.0+ bytes

========== DESCRIPTIVE STATISTICS ==========
           reg_no  attendance   internal  final_exam  study_hours
count   13.000000   12.000000  13.000000   12.000000    13.000000
mean   106.076923   90.833333  80.307692   83.416667     3.538462
std      3.774068   16.612336   7.792995    8.468748     1.126601
min    101.000000   75.000000  65.000000   68.000000     2.000000
25%    103.000000   83.750000  75.000000   77.250000     3.000000
50%    106.000000   89.000000  82.000000   86.000000     4.000000
75%    109.000000   91.250000  85.000000   88.750000     4.000000
max    112.000000  140.000000  90.000000   94.000000     5.000000

========== MISSING VALUES ==========
reg_no         0
department     0
gender         1
attendance     1
internal       0
```

```
final_exam     1
study_hours    0
dtype: int64

========== CLEANED DATASET ==========
    reg_no department  gender  attendance  internal  final_exam  study_hours
0      101       AIDS    Male        90.0        85        88.0            4
1      102        CSE  Female        85.0        80        82.0            3
2      103         IT    Male        75.0        70        72.0            2
3      104       AIDS  Female        95.0        90        94.0            5
4      105        CSE    Male        80.0        75        78.0            3
5      106         IT    Male        88.0        65        68.0            2
6      107       AIDS  Female        88.0        88        91.0            5
7      108        CSE    Male        88.0        82        85.0            4
8      109         IT  Female        92.0        79        86.0            3
9      110       AIDS    Male        78.0        72        75.0            2
10     111        CSE  Female        86.0        84        87.0            4
11     112         IT    Male        91.0        89        93.0            5

========== MISSING VALUES AFTER CLEANING ==========
reg_no         0
department     0
gender         0
attendance     0
internal       0
final_exam     0
study_hours    0
dtype: int64

Final Dataset Shape: (12, 7)

========== FINAL EXAM STATISTICS ==========
Mean: 83.25
Median: 85.5
Standard Deviation: 8.389658135845801
Minimum: 68.0
Maximum: 94.0

========== DEPARTMENT-WISE MEAN ==========
department
AIDS    87.00
CSE     83.00
IT      79.75
Name: final_exam, dtype: float64


========== CORRELATION ==========
Correlation: 0.9418458005432

Cleaned dataset saved successfully!
EXPERIMENT 1 COMPLETED SUCCESSFULLY!
```

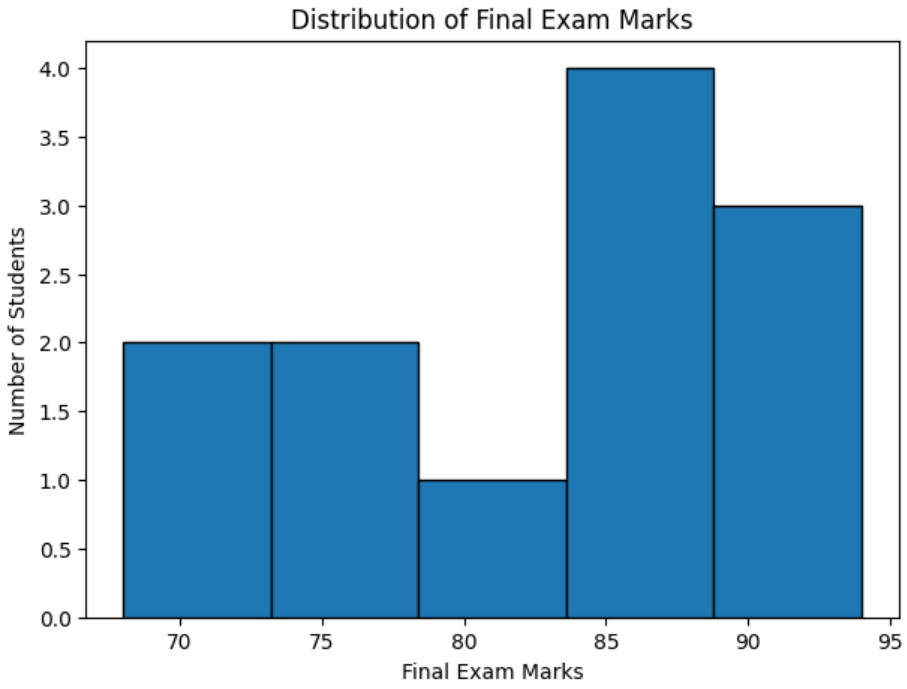


Figure 1.1: Plot/visualisation captured from the executed notebook.

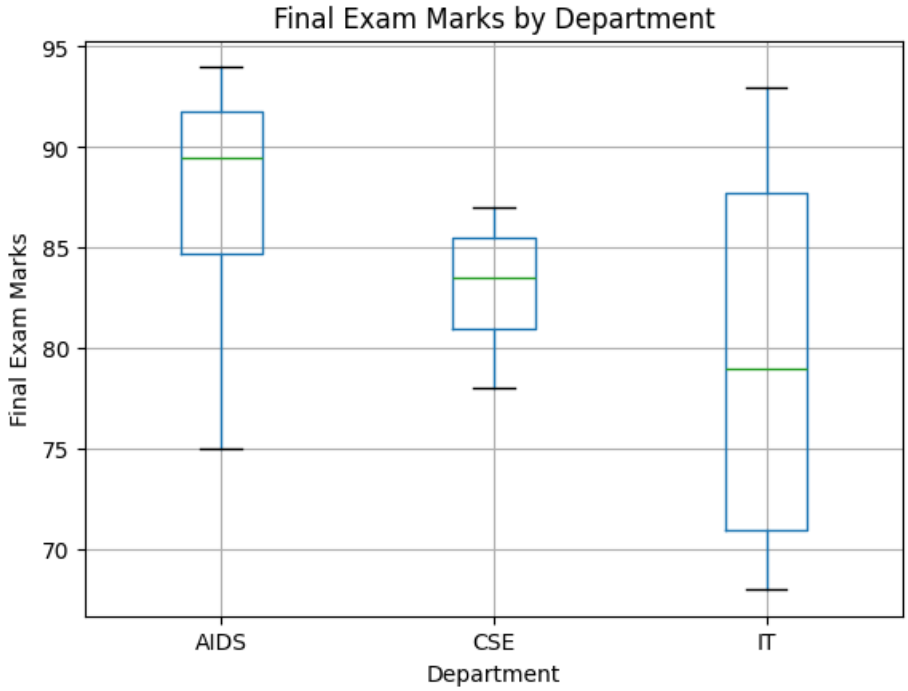


Figure 1.2: Plot/visualisation captured from the executed notebook.

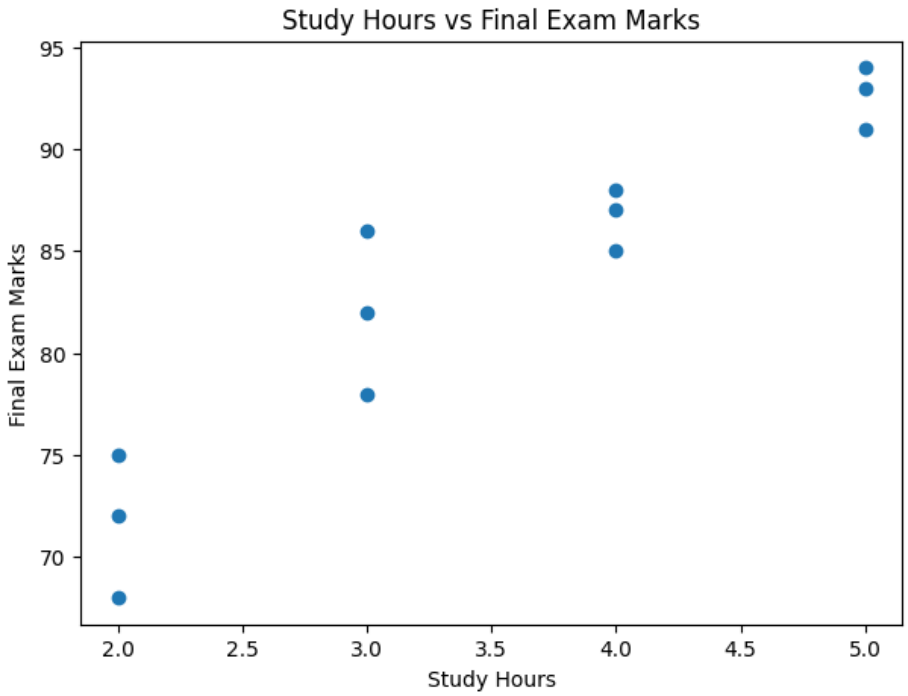


Figure 1.3: Plot/visualisation captured from the executed notebook.

Result / Conclusion

The notebook reports a dataset shape of 13 rows and 7 columns before cleaning. It identifies missing values in gender, attendance and final_exam. The cleaned dataset is displayed, and the recorded attendance/final-exam correlation is 0.9418458005432.

Experiment 2: Semantic Network and Inheritance Queries

Difficulty: Simple

Aim

To represent a small animal-domain knowledge base as a semantic network and answer inheritance queries through class and instance relationships.

Tools

Python, Google Colab, NetworkX, Matplotlib.

Procedure

7. Create a directed graph and add the Animal, Bird, Fish, Canary, Penguin and Tweety nodes.
8. Add is-a and instance-of relationships and attach properties to the relevant nodes.
9. Draw the semantic network with labelled relationships.
10. Implement inheritance lookup and query the properties of Tweety, Penguin and Fish.
11. Find the path from Tweety to Animal and inspect the returned relationships.

Python Code

```
# EXPERIMENT 2: SEMANTIC NETWORK AND INHERITANCE QUERIES

# 1. Import required libraries
import networkx as nx
import matplotlib.pyplot as plt

# 2. Create a directed graph
G = nx.DiGraph()

# 3. Add nodes (Classes and Individuals)
nodes = [
    "Animal", "Bird", "Fish",
    "Canary", "Penguin", "Tweety"
]

G.add_nodes_from(nodes)

# 4. Add relationships between nodes
G.add_edge("Bird", "Animal", relation="is-a")
G.add_edge("Fish", "Animal", relation="is-a")
G.add_edge("Canary", "Bird", relation="is-a")
G.add_edge("Penguin", "Bird", relation="is-a")
G.add_edge("Tweety", "Canary", relation="instance-of")

# 5. Add properties to nodes
properties = {
    "Animal": {"can": "move"},
    "Bird": {"can": "fly", "has-part": "wings"},
    "Fish": {"can": "swim"},
    "Canary": {"colour": "yellow"},
    "Penguin": {"can": "cannot fly"},
    "Tweety": {"colour": "yellow"}
}

nx.set_node_attributes(G, properties)

# 6. Display semantic network
plt.figure(figsize=(10, 7))

pos = nx.spring_layout(G, seed=42)

nx.draw(
    G, pos,
    with_labels=True,
    node_color="lightblue",
```

```
    node_size=3000,
    font_size=10,
    font_weight="bold",
    arrows=True
)

edge_labels = nx.get_edge_attributes(G, "relation")
nx.draw_networkx_edge_labels(
    G, pos,
    edge_labels=edge_labels,
    font_color="red"
)

plt.title("Semantic Network of Animals")
plt.show()

# 7. Inheritance function
def inherits(node, relation):
    current = node
    visited = set()

    while current is not None and current not in visited:
        visited.add(current)

        # Check local property first
        if relation in G.nodes[current]:
            return G.nodes[current][relation]

        # Find parent node
        parents = [
            neighbor for neighbor in G.successors(current)
            if G[current][neighbor]["relation"] in ["is-a", "instance-of"]
        ]

        current = parents[0] if parents else None

    return "Property not found"

# 8. Test inheritance queries
print("\n========== INHERITANCE QUERIES ==========")

print("Can Tweety fly?:", inherits("Tweety", "can"))
print("Does Tweety have wings?:", inherits("Tweety", "has-part"))
print("Can Penguin fly?:", inherits("Penguin", "can"))
print("What is Tweety's colour?:", inherits("Tweety", "colour"))
print("Can a Fish swim?:", inherits("Fish", "can"))

# 9. Find path between two concepts
print("\n========== PATH SEARCH ==========")

source = "Tweety"
target = "Animal"

path = nx.shortest_path(G, source, target)

print("Path from", source, "to", target, ":")
print(" -> ".join(path))

# 10. Display all relationships
print("\n========== SEMANTIC NETWORK RELATIONSHIPS ==========")

for u, v, data in G.edges(data=True):
    print(u, "--", data["relation"], "-->", v)

print("\nEXPERIMENT 2 COMPLETED SUCCESSFULLY!")
```

Actual Output from the Executed Notebook

```
========== INHERITANCE QUERIES ==========
Can Tweety fly?: fly
Does Tweety have wings?: wings
Can Penguin fly?: cannot fly
```

```
What is Tweety's colour?: yellow
Can a Fish swim?: swim

========== PATH SEARCH ==========
Path from Tweety to Animal :
Tweety -> Canary -> Bird -> Animal

========== SEMANTIC NETWORK RELATIONSHIPS ==========
Bird -- is-a --> Animal
Fish -- is-a --> Animal
Canary -- is-a --> Bird
Penguin -- is-a --> Bird
Tweety -- instance-of --> Canary

EXPERIMENT 2 COMPLETED SUCCESSFULLY!
```

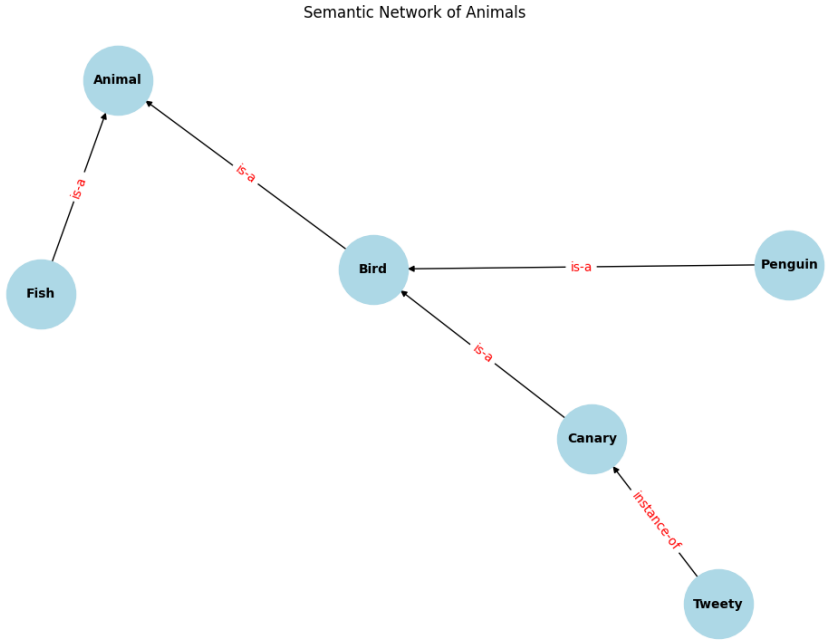


Figure 2.1: Plot/visualisation captured from the executed notebook.

Result / Conclusion

The notebook answers five inheritance queries and finds the path Tweety → Canary → Bird → Animal. It also displays the semantic network and its relationships.

Experiment 3: Frames and Inheritance

Difficulty: Simple

Aim

To implement a frame-based knowledge representation system with slots, default values, inheritance, a calculated value and a validation demon.

Tools

Python, Google Colab.

Procedure

12. Define a Frame class with a name, parent frame and slot dictionary.
13. Implement methods to set and retrieve slot values, including inherited lookup from a parent.
14. Create the Vehicle → Car → MyCar frame hierarchy and populate its slots.
15. Calculate mileage using the notebook's if-needed-style procedure.
16. Attempt to assign a negative mileage value to demonstrate the validation warning.

Python Code

```
# EXPERIMENT 3: FRAMES AND INHERITANCE

# 1. Create Frame class
class Frame:
    def __init__(self, name, parent=None):
        self.name = name
        self.parent = parent
        self.slots = {}

    # Add slot
    def set(self, slot, value):
        if slot == "mileage" and value < 0:
            print("Warning: Mileage cannot be negative!")
            return
        self.slots[slot] = value

    # Get slot value
    def get(self, slot):
        if slot in self.slots:
            return self.slots[slot]

        if self.parent:
            return self.parent.get(slot)

        return "Not Available"


# 2. Create frame hierarchy
Vehicle = Frame("Vehicle")
Vehicle.set("wheels", 4)
Vehicle.set("fuel", "Petrol")

Car = Frame("Car", Vehicle)
Car.set("brand", "Toyota")

MyCar = Frame("MyCar", Car)
MyCar.set("colour", "White")
MyCar.set("odometer", 15000)

# 3. If-needed procedure for mileage
def calculate_mileage(odometer, fuel_used):
    return odometer / fuel_used

MyCar.set("fuel_used", 100)
```

```
# 4. Display inherited and local values
print("========== FRAME INFORMATION ==========")

print("Vehicle Wheels:", MyCar.get("wheels"))
print("Fuel Type:", MyCar.get("fuel"))
print("Car Brand:", MyCar.get("brand"))
print("Car Colour:", MyCar.get("colour"))
print("Odometer:", MyCar.get("odometer"))

# Calculate mileage using procedure
mileage = calculate_mileage(
    MyCar.get("odometer"),
    MyCar.get("fuel_used")
)

print("Mileage:", mileage, "km/litre")

# 5. Demonstrate demon (if-added procedure)
print("\n========== DEMONSTRATION ==========")
MyCar.set("mileage", -10)

print("\nEXPERIMENT 3 COMPLETED SUCCESSFULLY!")
```

Actual Output from the Executed Notebook

```
========== FRAME INFORMATION ==========
Vehicle Wheels: 4
Fuel Type: Petrol
Car Brand: Toyota
Car Colour: White
Odometer: 15000
Mileage: 150.0 km/litre

========== DEMONSTRATION ==========
Warning: Mileage cannot be negative!

EXPERIMENT 3 COMPLETED SUCCESSFULLY!
```

Result / Conclusion

The recorded output inherits 4 wheels and Petrol fuel from Vehicle, Toyota from Car, and White colour and 15000 odometer from MyCar. The mileage calculation returns 150.0 km/litre, and the validation demon prints a warning for negative mileage.

Experiment 4: Rule-Based System with Forward Chaining

Difficulty: Simple

Aim

To build a rule-based system that derives new facts from an initial working memory using forward chaining.

Tools

Python, Google Colab.

Procedure

17. Represent IF–THEN rules as condition sets and conclusions.
18. Create the initial facts for salary, credit score and job stability.
19. Repeatedly identify rules whose conditions are satisfied by the current facts.
20. Add newly inferred facts to working memory and continue until no new facts can be generated.
21. Display the rule-firing cycles and final conclusions.

Python Code

```
# EXPERIMENT 4: FORWARD CHAINING

# 1. Define IF-THEN rules
rules = [
    ({"income_ok"}, "eligible_income"),
    ({"credit_good"}, "eligible_credit"),
    ({"eligible_income", "eligible_credit"}, "loan_approved"),
    ({"salary_high"}, "income_ok"),
    ({"credit_score_good"}, "credit_good"),
    ({"loan_approved"}, "low_risk"),
    ({"low_risk", "job_stable"}, "premium_customer"),
    ({"income_low"}, "loan_rejected")
]

# 2. Initial facts
facts = {"salary_high", "credit_score_good", "job_stable"}

# 3. Forward chaining function
def forward_chaining(facts, rules):
    facts = set(facts)
    cycle = 1

    while True:
        new_facts = []

        for conditions, conclusion in rules:
            if conditions.issubset(facts) and conclusion not in facts:
                new_facts.append(conclusion)

        if not new_facts:
            break

        print("\nCycle:", cycle)
        for fact in new_facts:
            print("Rule fired ->", fact)
            facts.add(fact)

        print("Current Facts:", facts)
        cycle += 1

    return facts

# 4. Execute forward chaining
print("Initial Facts:", facts)

result = forward_chaining(facts, rules)
```

```
# 5. Final conclusions
print("\n========== FINAL CONCLUSIONS ==========")

for fact in result:
    print(fact)

print("\nEXPERIMENT 4 COMPLETED SUCCESSFULLY!")
```

Actual Output from the Executed Notebook

```
Initial Facts: {'job_stable', 'salary_high', 'credit_score_good'}

Cycle: 1
Rule fired -> income_ok
Rule fired -> credit_good
Current Facts: {'salary_high', 'income_ok', 'job_stable', 'credit_good', 'credit_score_good'}

Cycle: 2
Rule fired -> eligible_income
Rule fired -> eligible_credit
Current Facts: {'eligible_income', 'salary_high', 'income_ok', 'job_stable', 'credit_good', 'eligible_credit',
'credit_score_good'}

Cycle: 3
Rule fired -> loan_approved
Current Facts: {'eligible_income', 'salary_high', 'income_ok', 'job_stable', 'loan_approved', 'credit_good',
'eligible_credit', 'credit_score_good'}

Cycle: 4
Rule fired -> low_risk
Current Facts: {'eligible_income', 'salary_high', 'income_ok', 'low_risk', 'job_stable', 'loan_approved',
'credit_good', 'eligible_credit', 'credit_score_good'}

Cycle: 5
Rule fired -> premium_customer
Current Facts: {'eligible_income', 'salary_high', 'income_ok', 'low_risk', 'job_stable', 'loan_approved',
'premium_customer', 'credit_good', 'eligible_credit', 'credit_score_good'}

========== FINAL CONCLUSIONS ==========
eligible_income
salary_high
income_ok
low_risk
job_stable
loan_approved
premium_customer
credit_good
eligible_credit
credit_score_good

EXPERIMENT 4 COMPLETED SUCCESSFULLY!
```

Result / Conclusion

The notebook runs five inference cycles and derives eligible_income, eligible_credit, loan_approved, low_risk and premium_customer from the initial facts.

Experiment 5: Backward Chaining with Why/How Explanation

Difficulty: Medium

Aim

To implement goal-driven reasoning by backward chaining and provide a trace explaining why a goal is being checked and how a conclusion is supported.

Tools

Python, Google Colab.

Procedure

22. Define rules for plant conditions such as yellow leaves, dry or wet soil, root rot and fungal disease.
23. Store known true and false observations in the fact base.
24. Implement a prove(goal) function that searches for rules that can establish the requested goal.
25. Ask for an unknown observation when the existing facts and rules cannot prove it.
26. Record the reasoning trace and display the conclusion and supporting rules.

Python Code

```
# EXPERIMENT 5: BACKWARD CHAINING

# 1. Define rules
rules = [
    (["yellow_leaves", "dry_soil"], "water_deficiency"),
    (["yellow_leaves", "wet_soil"], "root_rot"),
    (["brown_spots", "high_humidity"], "fungal_disease"),
    (["water_deficiency"], "plant_needs_water"),
    (["root_rot"], "plant_disease"),
    (["fungal_disease"], "plant_disease"),
    (["yellow_leaves", "brown_spots"], "serious_condition"),
    (["dry_soil", "wilting"], "water_deficiency"),
    (["wet_soil", "bad_smell"], "root_rot"),
    (["fungal_disease", "high_humidity"], "spread_risk"),
    (["plant_needs_water"], "watering_required"),
    (["plant_disease"], "treatment_required")
]

# 2. Store known facts
facts = {
    "yellow_leaves": True,
    "dry_soil": True,
    "brown_spots": False,
    "high_humidity": False,
    "wet_soil": False,
    "wilting": False,
    "bad_smell": False
}

# 3. Store explanation trace
trace = []

# 4. Backward chaining function
def prove(goal):
    if facts.get(goal) is True:
        return True

    if facts.get(goal) is False:
        return False

    for conditions, conclusion in rules:
        if conclusion == goal:
            print("\nWhy: Checking", goal)
            trace.append("Rule: " + str(conditions) + " -> " + goal)
```

```
            if all(prove(condition) for condition in conditions):
                return True

    # Ask user for unknown facts
    answer = input("Is " + goal + " present? (yes/no/why): ").lower()

    if answer == "why":
        print("Because it is required to prove:", goal)
        return prove(goal)

    facts[goal] = (answer == "yes")
    return facts[goal]

# 5. Test backward chaining
goal = "plant_disease"

print("========== BACKWARD CHAINING ==========")

if prove(goal):
    print("\nConclusion: Plant disease detected.")
else:
    print("\nConclusion: Plant disease not detected.")

# 6. How explanation
print("\n========== HOW EXPLANATION ==========")

for step in trace:
    print(step)

print("\nEXPERIMENT 5 COMPLETED SUCCESSFULLY!")
```

Actual Output from the Executed Notebook

```
========== BACKWARD CHAINING ==========

Why: Checking plant_disease

Why: Checking root_rot

Why: Checking root_rot
Is root_rot present? (yes/no/why): yes

Conclusion: Plant disease detected.

========== HOW EXPLANATION ==========
Rule: ['root_rot'] -> plant_disease
Rule: ['yellow_leaves', 'wet_soil'] -> root_rot
Rule: ['wet_soil', 'bad_smell'] -> root_rot

EXPERIMENT 5 COMPLETED SUCCESSFULLY!
```

Result / Conclusion

The saved interaction checks whether plant_disease can be proved, asks whether root_rot is present, receives the recorded answer 'yes', and concludes: Plant disease detected. The How trace lists the rules considered for root_rot and plant_disease.

Experiment 6: Building and Reasoning over an Ontology

Difficulty: Medium

Aim

To create a university-domain ontology with classes, properties and individuals, then run a reasoner to inspect the ontology's inferred information.

Tools

Python, Google Colab, Owlready2 and the HermiT reasoner.

Procedure

27. Install and import Owlready2 and create the university ontology.
28. Define Person, Student, Faculty, Course and Department classes.
29. Define object properties and a data property with the domains and ranges used in the notebook.
30. Create student, faculty, course and department individuals and connect them through properties.
31. Declare Student and Faculty disjoint and run the HermiT reasoner.
32. Display the ontology classes, individuals and reasoning results, then save the ontology.

Python Code

```
# EXPERIMENT 6: BUILDING AND REASONING OVER AN ONTOLOGY
!pip install owlready2
from owlready2 import *

# 1. Create ontology
onto = get_ontology("http://example.org/university.owl")

with onto:

    # 2. Create classes
    class Person(Thing):
        pass

    class Student(Person):
        pass

    class Faculty(Person):
        pass

    class Course(Thing):
        pass

    class Department(Thing):
        pass

    # 3. Create object properties
    class enrolledIn(ObjectProperty):
        domain = [Student]
        range = [Course]

    class teaches(ObjectProperty):
        domain = [Faculty]
        range = [Course]

    class belongsTo(ObjectProperty):
        domain = [Person]
        range = [Department]

    # 4. Create data property
    class hasName(DataProperty):
        range = [str]

    # 5. Create individuals
```

```
    s1 = Student("Student1")
    f1 = Faculty("Faculty1")
    c1 = Course("Python")
    d1 = Department("AIDS")

    s1.hasName = ["Arun"]
    f1.hasName = ["Kumar"]

    s1.enrolledIn = [c1]
    f1.teaches = [c1]
    s1.belongsTo = [d1]

    # 6. Define disjoint classes
    AllDisjoint([Student, Faculty])

# 7. Display ontology information
print("========== ONTOLOGY ==========")

print("Classes:", list(onto.classes()))

print("\nIndividuals:", list(onto.individuals()))

print("\nStudent Name:", s1.hasName)
print("Enrolled Course:", s1.enrolledIn)
print("Faculty Teaching:", f1.teaches)

# 8. Run reasoner
with onto:
    sync_reasoner(infer_property_values=True)

# 9. Display inferred facts
print("\n========== REASONING RESULTS ==========")

for individual in onto.individuals():
    print(individual.name, ":", list(individual.is_a))

# 10. Save ontology
onto.save(file="university.owl", format="rdfxml")

print("\nOntology saved successfully!")
print("EXPERIMENT 6 COMPLETED SUCCESSFULLY!")
```

Actual Output from the Executed Notebook

```
* Owlready2 * Running HermiT...
    java -Xmx2000M -cp /usr/local/lib/python3.13/dist-packages/owlready2/hermit:/usr/local/lib/python3.13/dist-
packages/owlready2/hermit/HermiT.jar org.semanticweb.HermiT.cli.CommandLine -c -O -D -I file:////tmp/tmpfhm3_104 -Y


========== REASONING RESULTS ==========
Student1 : [university.Student]
Faculty1 : [university.Faculty]
Python : [university.Course]
AIDS : [university.Department]

Ontology saved successfully!
EXPERIMENT 6 COMPLETED SUCCESSFULLY!

* Owlready2 * HermiT took 1.3638403415679932 seconds
* Owlready * (NB: only changes on entities loaded in Python are shown, other changes are done but not listed)
```

Result / Conclusion

The notebook displays five classes and four individuals: Student1, Faculty1, Python and AIDS. The reasoner returns their declared classes, and the ontology is saved successfully.

Experiment 7: Building and Querying a Knowledge Graph

Difficulty: Medium

Aim

To construct an RDF knowledge graph from student information, query it using SPARQL and analyse its structure as a NetworkX graph.

Tools

Python, Google Colab, RDFLib, NetworkX, Matplotlib.

Procedure

33. Create an RDF graph and define the example namespace.
34. Add student, department, city and project relationships as RDF triples.
35. Print the graph triples and execute the SPARQL query for project assignments.
36. Query the people belonging to the AIDS department.
37. Convert the RDF relationships into a NetworkX graph, calculate node degrees and visualise the graph.

Python Code

```
# EXPERIMENT 7: BUILDING AND QUERYING A KNOWLEDGE GRAPH

import networkx as nx
import matplotlib.pyplot as plt
from rdflib import Graph, Namespace, Literal, RDF

# 1. Create knowledge graph
g = Graph()
EX = Namespace("http://example.org/")

# 2. Add student information (using underscores instead of spaces to ensure valid URIs)
students = [
    ("Arun", "AIDS", "Chennai", "Project_Alpha"),
    ("Kumar", "CSE", "Madurai", "Project_Beta"),
    ("Priya", "AIDS", "Chennai", "Project_Gamma"),
    ("Arun", "AIDS", "Chennai", "Project_Alpha")
]

for name, dept, city, project in students:
    person = EX[name]
    g.add((person, RDF.type, EX.Person))
    g.add((person, EX.memberOf, EX[dept]))
    g.add((person, EX.livesIn, EX[city]))
    g.add((person, EX.worksOn, EX[project]))

# 3. Display knowledge graph triples
print("========== KNOWLEDGE GRAPH ==========")

for triple in g:
    print(triple)

# 4. SPARQL query
query = """
SELECT ?person ?project
WHERE {
    ?person <http://example.org/worksOn> ?project .
}
"""

print("\n========== SPARQL QUERY RESULTS ==========")

for row in g.query(query):
    print(row.person.split("/")[-1],
          "works on",
          row.project.split("/")[-1])
```

```
# 5. Find people in AIDS department
query2 = """
SELECT ?person
WHERE {
    ?person <http://example.org/memberOf>
    <http://example.org/AIDS> .
}
"""

print("\n========== AIDS DEPARTMENT ==========")

for row in g.query(query2):
    print(row.person.split("/")[-1])

# 6. Convert RDF graph to NetworkX
G = nx.DiGraph()

for s, p, o in g:
    G.add_edge(str(s).split("/")[-1],
               str(o).split("/")[-1],
               relation=str(p).split("/")[-1])

# 7. Calculate node degree
print("\n========== NODE DEGREE ==========")

for node, degree in G.degree():
    print(node, ":", degree)

# 8. Draw knowledge graph
plt.figure(figsize=(10, 7))

pos = nx.spring_layout(G, seed=42)

nx.draw(G, pos, with_labels=True,
        node_color="lightblue",
        node_size=2000,
        font_weight="bold",
        arrows=True)

plt.title("Student Knowledge Graph")
plt.show()

# 9. Save graph
g.serialize("knowledge_graph.ttl", format="turtle")

print("\nKnowledge graph saved successfully!")
print("EXPERIMENT 7 COMPLETED SUCCESSFULLY!")
```

Actual Output from the Executed Notebook

```
========== KNOWLEDGE GRAPH ==========

========== SPARQL QUERY RESULTS ==========
Arun works on Project_Alpha
Kumar works on Project_Beta
Priya works on Project_Gamma

========== AIDS DEPARTMENT ==========
Arun
Priya

========== NODE DEGREE ==========
Priya : 4
Person : 3
Project_Gamma : 1
Arun : 4
Project_Alpha : 1
AIDS : 2
Kumar : 4
Project_Beta : 1
```

```
Chennai : 2
Madurai : 1
CSE : 1


Knowledge graph saved successfully!
EXPERIMENT 7 COMPLETED SUCCESSFULLY!
```

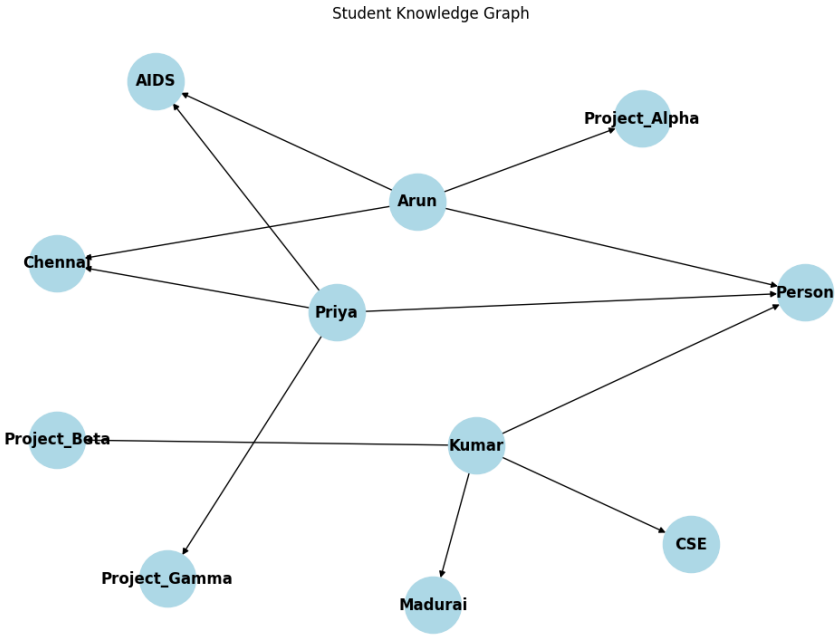


Figure 7.1: Plot/visualisation captured from the executed notebook.

Result / Conclusion

The notebook returns three project assignments (Arun–Project_Alpha, Kumar–Project_Beta and Priya–Project_Gamma), identifies Arun and Priya in AIDS, and prints node-degree values. A graph visualisation is saved in the notebook.

Experiment 8: Hypothesis Testing, ANOVA and Chi-Square

Difficulty: Medium

Aim

To apply common statistical hypothesis tests to a student-score dataset and interpret the test statistics, p-values and confidence interval.

Tools

Python, Google Colab, Pandas, NumPy, SciPy, Matplotlib.

Procedure

38. Create the 60-record dataset containing method, before/after scores, gender and result.
39. Perform a one-sample t-test, independent t-test and paired t-test.
40. Compare the three methods using one-way ANOVA.
41. Create a contingency table and perform a chi-square test.
42. Use the recorded p-values to report significance decisions at the 0.05 level.
43. Plot the score distribution and calculate the 95% confidence interval.

Python Code

```
# EXPERIMENT 8: HYPOTHESIS TESTING, ANOVA AND CHI-SQUARE

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# 1. Create sample dataset
data = {
    'method': ['A']*20 + ['B']*20 + ['C']*20,
    'score_before': np.random.randint(40, 60, 60),
    'score_after': np.concatenate([
        np.random.randint(65, 80, 20),
        np.random.randint(70, 85, 20),
        np.random.randint(75, 95, 20)
    ]),
    'gender': ['Male', 'Female'] * 30,
    'result': ['Pass']*45 + ['Fail']*15
}

df = pd.DataFrame(data)

print("========== DATASET ==========")
print(df.head())

# 2. One-sample t-test
t1, p1 = stats.ttest_1samp(df['score_after'], 60)

print("\nOne Sample T-Test")
print("Statistic:", t1)
print("P-value:", p1)

# 3. Independent t-test
a = df[df['method'] == 'A']['score_after']
b = df[df['method'] == 'B']['score_after']

t2, p2 = stats.ttest_ind(a, b)

print("\nIndependent T-Test")
print("Statistic:", t2)
print("P-value:", p2)

# 4. Paired t-test
```

```
t3, p3 = stats.ttest_rel(
    df['score_before'],
    df['score_after']
)

print("\nPaired T-Test")
print("Statistic:", t3)
print("P-value:", p3)

# 5. One-way ANOVA
c = df[df['method'] == 'C']['score_after']

f, p4 = stats.f_oneway(a, b, c)

print("\nANOVA Test")
print("F-statistic:", f)
print("P-value:", p4)

# 6. Chi-square test
table = pd.crosstab(df['gender'], df['result'])

chi, p5, dof, expected = stats.chi2_contingency(table)

print("\nChi-Square Test")
print("Chi-square:", chi)
print("P-value:", p5)

# 7. Display decisions
print("\n========== TEST DECISIONS ==========")

for name, p in [
    ("One Sample T-Test", p1),
    ("Independent T-Test", p2),
    ("Paired T-Test", p3),
    ("ANOVA", p4),
    ("Chi-Square", p5)
]:
    print(name, ":",
          "Significant" if p < 0.05 else "Not Significant")

# 8. Box plot
df.boxplot(column='score_after', by='method')
plt.title("Exam Scores by Teaching Method")
plt.suptitle("")
plt.xlabel("Teaching Method")
plt.ylabel("Score")
plt.show()

# 9. 95% Confidence Interval
mean = df['score_after'].mean()
sem = stats.sem(df['score_after'])

ci = stats.t.interval(
    0.95,
    len(df)-1,
    loc=mean,
    scale=sem
)

print("\n95% Confidence Interval:", ci)

print("\nEXPERIMENT 8 COMPLETED SUCCESSFULLY!")
```

Actual Output from the Executed Notebook

```
========== DATASET ==========
  method  score_before  score_after  gender result
0      A            44           77    Male   Pass
1      A            46           72  Female   Pass
2      A            44           71    Male   Pass
3      A            45           70  Female   Pass
4      A            41           69    Male   Pass
```

```
One Sample T-Test
Statistic: 18.164820483720476
P-value: 7.709866913095401e-26

Independent T-Test
Statistic: -4.752454898166987
P-value: 2.8679410449723364e-05

Paired T-Test
Statistic: -24.042485428899944
P-value: 3.566554559353097e-32

ANOVA Test
F-statistic: 45.293075749056165
P-value: 1.677388093358709e-12

Chi-Square Test
Chi-square: 0.0
P-value: 1.0

========== TEST DECISIONS ==========
One Sample T-Test : Significant
Independent T-Test : Significant
Paired T-Test : Significant
ANOVA : Significant
Chi-Square : Not Significant


95% Confidence Interval: (np.float64(75.42393269169487), np.float64(79.24273397497178))

EXPERIMENT 8 COMPLETED SUCCESSFULLY!
```

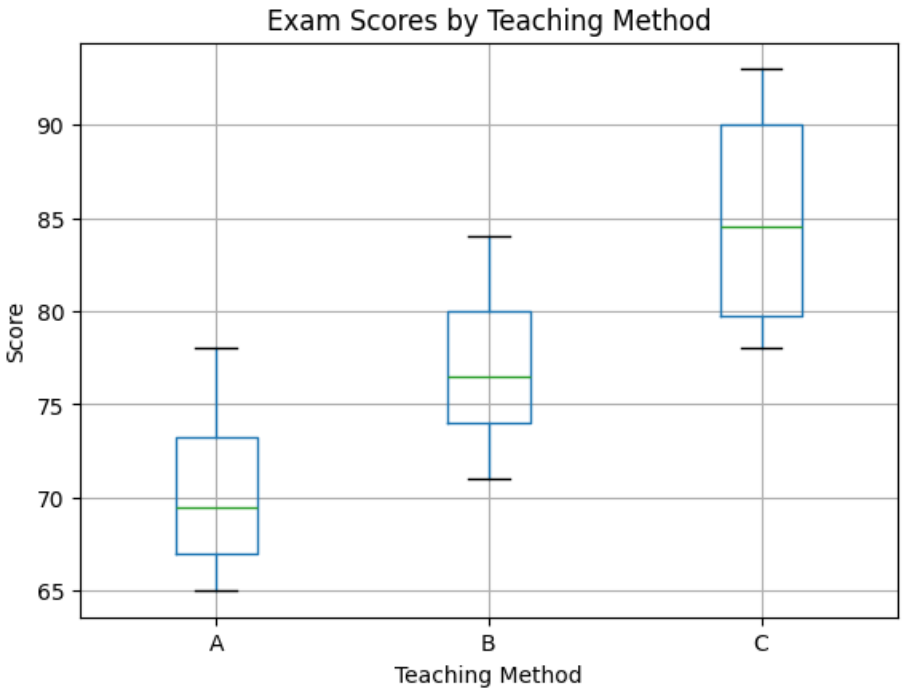


Figure 8.1: Plot/visualisation captured from the executed notebook.

Result / Conclusion

The saved run reports significant results for the one-sample, independent and paired t-tests and ANOVA, while the chi-square test is not significant. The recorded 95% confidence interval is approximately 75.4239 to 79.2427. Because the notebook creates random scores without setting a random seed, rerunning it may produce different values.

Experiment 9: Decision Tree Rule Learning

Difficulty: Hard

Aim

To train and evaluate decision-tree classifiers at different depths, compare them with a majority-class baseline and inspect the learned rules and classification metrics.

Tools

Python, Google Colab, NumPy, Pandas, Matplotlib, Scikit-learn.

Procedure

44. Load the Breast Cancer dataset and separate features and labels.
45. Split the data into training and testing sets using a stratified 70:30 split.
46. Calculate the majority-class baseline accuracy.
47. Train decision trees with maximum depths 2, 3, 4 and 6 and record training, testing and five-fold cross-validation accuracy.
48. Select the depth with the best cross-validation score and print the learned rules.
49. Evaluate the selected model using the confusion matrix and classification report.

Python Code

```
# EXPERIMENT 9: DECISION TREE RULE LEARNING

# 1. Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 2. Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# 3. Split dataset (70:30)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# 4. Baseline model
baseline = np.full(len(y_test), np.bincount(y_train).argmax())
print("Baseline Accuracy:", accuracy_score(y_test, baseline))

# 5. Train decision trees with different depths
depths = [2, 3, 4, 6]
train_scores = []
test_scores = []
cv_scores = []

for depth in depths:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)

    train_scores.append(accuracy_score(y_train, model.predict(X_train)))
    test_scores.append(accuracy_score(y_test, model.predict(X_test)))

    cv = cross_val_score(model, X_train, y_train, cv=5)
    cv_scores.append(cv.mean())
```

```
    print("\nTree Depth:", depth)
    print("Training Accuracy:", train_scores[-1])
    print("Testing Accuracy:", test_scores[-1])
    print("Cross Validation:", cv_scores[-1])

# 6. Select best depth using cross-validation
best_depth = depths[np.argmax(cv_scores)]

model = DecisionTreeClassifier(max_depth=best_depth, random_state=42)
model.fit(X_train, y_train)

# 7. Display learned IF-THEN rules
print("\n========== DECISION TREE RULES ==========")
print(export_text(model, feature_names=list(data.feature_names)))

# 8. Model evaluation
y_pred = model.predict(X_test)

print("\n========== EVALUATION ==========")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n",
      classification_report(y_test, y_pred, target_names=data.target_names))

# 9. Plot accuracy against tree depth
plt.plot(depths, train_scores, marker='o', label='Training Accuracy')
plt.plot(depths, test_scores, marker='o', label='Testing Accuracy')
plt.plot(depths, cv_scores, marker='o', label='Cross Validation')

plt.xlabel("Tree Depth")
plt.ylabel("Accuracy")
plt.title("Decision Tree Accuracy vs Depth")
plt.legend()
plt.grid()
plt.show()

print("\nBest Tree Depth:", best_depth)
print("\nEXPERIMENT 9 COMPLETED SUCCESSFULLY!")
```

Actual Output from the Executed Notebook

```
Baseline Accuracy: 0.6257309941520468

Tree Depth: 2
Training Accuracy: 0.964824120603015
Testing Accuracy: 0.9181286549707602
Cross Validation: 0.9398101265822785

Tree Depth: 3
Training Accuracy: 0.9798994974874372
Testing Accuracy: 0.9239766081871345
Cross Validation: 0.9424050632911392

Tree Depth: 4
Training Accuracy: 0.992462311557789
Testing Accuracy: 0.9239766081871345
Cross Validation: 0.9347784810126581

Tree Depth: 6
Training Accuracy: 1.0
Testing Accuracy: 0.9181286549707602
Cross Validation: 0.934873417721519

========== DECISION TREE RULES ==========
|--- worst radius <= 16.80
|   |--- worst concave points <= 0.14
|   |   |--- smoothness error <= 0.00
|   |   |   |--- class: 1
|   |   |--- smoothness error >  0.00
|   |   |   |--- class: 1
|   |--- worst concave points >  0.14
```

```
|   |   |--- worst texture <= 23.74
|   |   |   |--- class: 1
|   |   |--- worst texture >  23.74
|   |   |   |--- class: 0
|--- worst radius >  16.80
|   |--- texture error <= 0.45
|   |   |--- class: 1
|   |--- texture error >  0.45
|   |   |--- worst concavity <= 0.19
|   |   |   |--- class: 1
|   |   |--- worst concavity >  0.19
|   |   |   |--- class: 0


========== EVALUATION ==========
Accuracy: 0.9239766081871345
Confusion Matrix:
 [[ 56   8]
 [  5 102]]

Classification Report:
              precision    recall  f1-score   support

   malignant       0.92      0.88      0.90        64
      benign       0.93      0.95      0.94       107

    accuracy                           0.92       171
   macro avg       0.92      0.91      0.92       171
weighted avg       0.92      0.92      0.92       171


Best Tree Depth: 3

EXPERIMENT 9 COMPLETED SUCCESSFULLY!
```

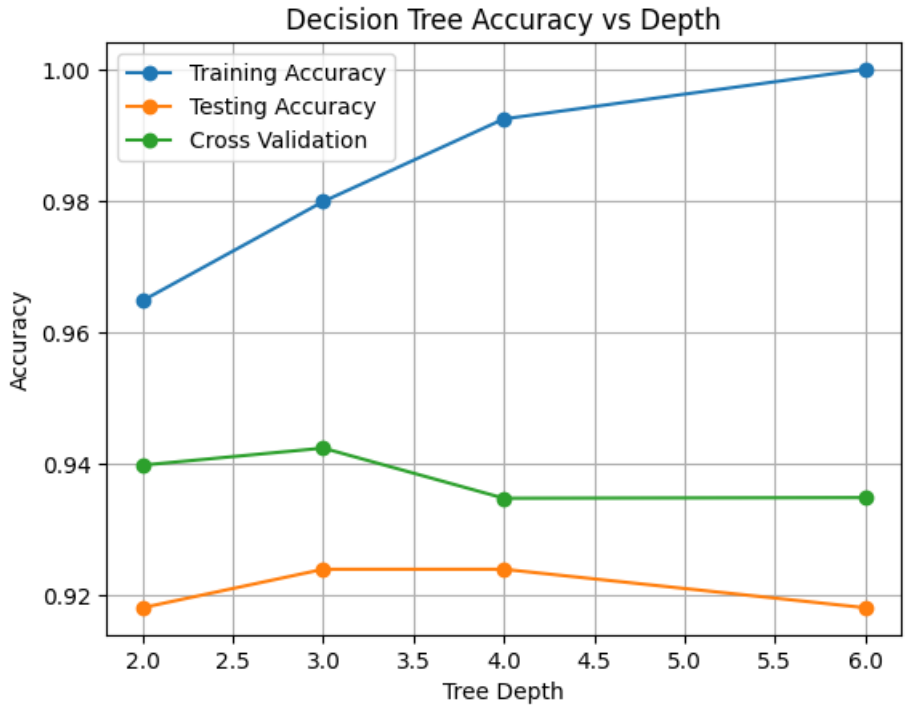


Figure 9.1: Plot/visualisation captured from the executed notebook.

Result / Conclusion

The recorded best depth is 3, with testing accuracy 0.9239766 and five-fold cross-validation accuracy 0.9424051. The confusion matrix is [[56, 8], [5, 102]]. The classification report gives malignant precision 0.92, recall 0.88 and F1-score 0.90; benign precision 0.93, recall 0.95 and F1-score 0.94.

Experiment 10: Hybrid Decision-Support System

Difficulty: Hard

Aim

To demonstrate a hybrid decision-support approach that combines a knowledge graph, expert rules and a machine-learning model to recommend a course.

Tools

Python, Google Colab, Pandas, NumPy, NetworkX, Scikit-learn.

Procedure

50. Create the student feature dataset containing mathematics, programming and interest values.
51. Train a decision-tree model and calculate cross-validation accuracy.
52. Build a small knowledge graph connecting student attributes with course concepts.
53. Define expert rules that produce a recommendation and certainty value.
54. Combine the rule recommendation and machine-learning prediction using the notebook's decision logic.
55. Display the final recommendation and the facts used to support it.

Python Code

```
# EXPERIMENT 10: HYBRID DECISION-SUPPORT SYSTEM

# 1. Import libraries
import pandas as pd
import networkx as nx
import numpy as np

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
from sklearn.metrics import accuracy_score

# 2. Create sample student dataset
data = {
    'math': [90, 80, 40, 85, 35, 75, 95, 45, 88, 50,
             92, 30, 78, 42, 89, 65, 55, 82, 38, 96],
    'programming': [85, 90, 35, 80, 40, 88, 95, 45, 92, 50,
                    89, 30, 82, 38, 94, 70, 55, 86, 42, 98],
    'interest': [1, 1, 0, 1, 0, 1, 1, 0, 1, 0,
                 1, 0, 1, 0, 1, 1, 0, 1, 0, 1],
    'course': [1, 1, 0, 1, 0, 1, 1, 0, 1, 0,
               1, 0, 1, 0, 1, 1, 0, 1, 0, 1]
}

df = pd.DataFrame(data)

X = df[['math', 'programming', 'interest']]
y = df['course']

# 3. Train Machine Learning model
model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X, y)

cv = cross_val_score(model, X, y, cv=5)
print("ML Cross Validation Accuracy:", cv.mean())

# 4. Create Knowledge Graph
G = nx.DiGraph()

G.add_edge("Student", "Mathematics")
G.add_edge("Student", "Programming")
G.add_edge("Programming", "AI Course")
G.add_edge("Mathematics", "Data Science Course")
```

```
print("\nKnowledge Graph Nodes:", list(G.nodes()))
print("Knowledge Graph Edges:", list(G.edges()))

# 5. Define expert rules
def expert_rules(math, programming, interest):
    if math >= 70 and programming >= 70 and interest == 1:
        return "AI Course", 0.9
    elif math >= 70:
        return "Data Science Course", 0.8
    else:
        return "Basic Programming Course", 0.7

# 6. Hybrid decision system
math = 85
programming = 90
interest = 1

rule_result, certainty = expert_rules(
    math, programming, interest
)

ml_result = model.predict(
    pd.DataFrame([[math, programming, interest]],
                 columns=X.columns)
)[0]

# 7. Combine Rules and ML
if ml_result == 1 and rule_result == "AI Course":
    recommendation = "AI Course"
elif ml_result == 1:
    recommendation = rule_result
else:
    recommendation = "Basic Programming Course"

# 8. Display explanation
print("\n========== HYBRID RECOMMENDATION ==========")
print("Rule Recommendation:", rule_result)
print("Rule Certainty:", certainty)
print("ML Prediction:", ml_result)
print("Final Recommendation:", recommendation)

print("\nFacts Used:")
print("Mathematics:", math)
print("Programming:", programming)
print("Interest:", interest)

print("\nEXPERIMENT 10 COMPLETED SUCCESSFULLY!")
```

Actual Output from the Executed Notebook

```
ML Cross Validation Accuracy: 1.0

Knowledge Graph Nodes: ['Student', 'Mathematics', 'Programming', 'AI Course', 'Data Science Course']
Knowledge Graph Edges: [('Student', 'Mathematics'), ('Student', 'Programming'), ('Mathematics', 'Data Science
Course'), ('Programming', 'AI Course')]

========== HYBRID RECOMMENDATION ==========
Rule Recommendation: AI Course
Rule Certainty: 0.9
ML Prediction: 1
Final Recommendation: AI Course

Facts Used:
Mathematics: 85
Programming: 90
Interest: 1

EXPERIMENT 10 COMPLETED SUCCESSFULLY!
```

Result / Conclusion

The recorded run reports ML cross-validation accuracy of 1.0. For the displayed case, the rule recommendation is AI Course with certainty 0.9, the ML prediction is 1, and the final recommendation is AI Course. The facts shown are Mathematics 85, Programming 90 and Interest 1. The notebook uses 20 sample records; this is smaller than the 100+ records specified in the lab manual.